

Analisi Tecnica delle Principali Categorie di Malware

NECKER

16 Novembre 2025

Indice

1	Introduzione al Malware	2
2	Malware a Replicazione Autonoma	2
2.1	Virus	2
2.1.1	Caso di Studio: CIH (Virus Chernobyl)	2
2.2	Worm (Verme)	3
2.2.1	Botnet, Zombie e Architettura C&C	3
3	Tecniche di Offuscamento e Anti-Analisi	3
3.1	Virus Polimorfi	4
3.2	Virus Metamorfi	4
3.3	Difficoltà del Reverse Engineering del Malware	4
3.3.1	Packing e Crittografia (Runtime Packers)	4
3.3.2	Tecniche Anti-Debugging	5
3.3.3	Tecniche Anti-VM / Anti-Sandbox	5
4	Malware basati sul Payload	5
4.1	Trojan (Cavallo di Troia)	5
4.2	Backdoor	6
4.3	Rootkit	6
4.4	Ransomware	6
4.4.1	Caso di Studio: WannaCry	7
4.5	Spyware e Keylogger	7
4.6	Logic Bomb (Bomba Logica)	8
5	Malware Moderni: Fileless	8

1 Introduzione al Malware

Il termine **Malware** (abbreviazione di *Malicious Software*) si riferisce a qualsiasi software progettato con l'intento di causare danni, interrompere il servizio, sottrarre dati sensibili o ottenere accesso non autorizzato a sistemi informatici.

Dal punto di vista tecnico, un malware è un programma che esegue azioni contrarie agli interessi dell'utente o del proprietario del sistema. La sua classificazione non è sempre netta, poiché molte minacce moderne sono **ibride** (ad esempio, un Worm che installa un Trojan, che a sua volta scarica un Ransomware).

Le categorie principali si distinguono in base a due criteri fondamentali:

- **Vettore di Propagazione:** Come il malware si diffonde (ad es. replicazione autonoma, interazione umana).
- **Payload:** L'azione malevola che il malware esegue una volta attivato (ad es. furto di dati, cifratura, occultamento).

2 Malware a Replicazione Autonoma

Questa categoria include software progettati specificamente per diffondersi.

2.1 Virus

Un **Virus** è un frammento di codice che **richiede un file host** per propagarsi. Non è un programma *standalone* (indipendente). Si "aggancia" a un eseguibile legittimo (es. `.exe`, `.dll`, script) o a un settore di avvio (Boot Sector).

- **Ciclo di vita:** Il virus rimane dormiente finché il file host non viene eseguito. A quel punto, il codice virale viene eseguito per primo, cerca altri file host da infettare e, infine, passa il controllo al programma legittimo (per non destare sospetti).
- **Infezione:** Può avvenire in vari modi:
 - *Appending/Prepending:* Aggiunge il suo codice all'inizio o alla fine del file host.
 - *Cavity (Spazio vuoto):* Si inserisce negli spazi inutilizzati (es. padding) all'interno delle sezioni del file PE (Portable Executable).
 - *Entry-Point Obfuscation (EPO):* Un programma ha nel suo header l'indirizzo della prima istruzione da eseguire, invece di modificare questo indirizzo (sarebbe facile detectare il virus), Modifica l'entry point del PE per puntare a sé stesso (cioè mette una jump al posto della prima istruzione), ma lo fa in modo offuscato per eludere l'analisi statica ad esempio rendendo il codice estremamente difficile da leggere, pieno di salti qua e là spesso inutili.

2.1.1 Caso di Studio: CIH (Virus Chernobyl)

CIH (conosciuto come "Chernobyl") è un esempio lampante di virus altamente distruttivo (attivo alla fine degli anni '90).

- **Tipo:** Virus infettore di file PE (Windows 9x).
- **Tecnica:** Sfruttava le "cavità" nei file eseguibili, rendendo difficile l'individuazione tramite il semplice controllo della dimensione del file.

- **Payload:** Il 26 Aprile (anniversario del disastro di Chernobyl), il virus si attivava. Il suo payload era duplice e catastrofico:
 1. Sovrascriveva il primo megabyte dell'hard disk (sovrascrivendo MBR (**Master Boot Record**: è uno schema di partizionamento che definisce come è suddiviso il disco e contiene il codice iniziale per avviare il sistema operativo) e FAT (**File Allocation Table**: è un file system che gestisce l'organizzazione dei file all'interno di una singola partizione)), rendendo il sistema inavviabile e i dati irrecuperabili.
 2. Tentava di sovrascrivere il **Flash BIOS** della scheda madre. Se riusciva (su chipset compatibili), rendeva l'hardware inutilizzabile (richiedendo la riprogrammazione fisica del chip).

2.2 Worm (Verme)

A differenza di un virus, un **Worm** è un programma *standalone* che si replica attivamente attraverso la rete. Non necessita di un file host o di interazione umana (dopo l'infezione iniziale) per diffondersi.

- **Propagazione:** Sfrutta vulnerabilità note (ad esempio, buffer overflow in un servizio di rete) o meccanismi di trasporto (email, condivisioni di rete) per copiare sé stesso su altri sistemi e auto-eseguirsi.
- **Impatto:** Il suo obiettivo primario è la diffusione. L'impatto principale è la saturazione della larghezza di banda di rete. Tuttavia, i worm moderni trasportano quasi sempre un payload secondario (come un backdoor o un ransomware).

2.2.1 Botnet, Zombie e Architettura C&C

L'uso più comune dei worm oggi è la creazione di **Botnet**.

- **Zombie (o Bot):** È un sistema infettato dal worm. Il sistema è ora sotto il controllo di un operatore remoto (il *bot-herder*).
- **Botnet:** È l'intera rete di computer "zombie" infetti.
- **Command & Control (C&C o C2):** È l'infrastruttura (uno o più server) utilizzata dal bot-herder per inviare comandi alla botnet.

Le architetture C&C possono essere:

- **Centralizzate (IRC/HTTP):** Tutti i bot si connettono a un server o a un canale IRC specifico. Facile da gestire, ma vulnerabile (se il server C&C viene smantellato, la botnet muore).
- **Decentralizzate (P2P):** I bot comunicano tra loro, scambiandosi comandi. Molto più resilienti al *takedown*.

Le botnet vengono "affittate" per compiere attività malevole su larga scala, come attacchi **DDoS** (Distributed Denial of Service), invio di spam/phishing, mining di criptovalute o proxy per altre attività illecite.

3 Tecniche di Offuscamento e Anti-Analisi

Per evitare il rilevamento da parte di software antivirus (AV) basati su firme (signature) e per complicare l'analisi da parte dei ricercatori, i malware utilizzano tecniche di offuscamento avanzate.

3.1 Virus Polimorfi

Un virus (o malware) polimorfo altera il proprio codice per apparire diverso ad ogni infezione, pur mantenendo la stessa funzionalità.

- **Meccanismo:** Il corpo principale del malware (*payload*) viene cifrato con una chiave di volta in volta diversa.
- **Motore Polimorfo:** Il malware è composto da due parti:
 1. Il **payload cifrato**.
 2. Una piccola **routine di decriptazione** (chiamata *decryption stub* o *loader*) che viene eseguita per prima.
- **Mutazione:** Ad ogni nuova infezione, il motore polimorfo genera una nuova chiave, ricifra il payload e, soprattutto, **muta la routine di decriptazione** (es. inserendo istruzioni NOP, cambiando le istruzioni usate, riordinando il codice).
- **Rilevamento:** Inefficace contro le firme statiche (perché il payload è cifrato e lo stub cambia). Richiede **euristica** o **emulazione** (l'AV esegue il codice in una sandbox, attende che si decifri e analizza il payload in memoria).

3.2 Virus Metamorfi

Il metamorfismo è un'evoluzione del polimorfismo. Un malware metamorfico **non cifra il proprio corpo**, ma **riscrive completamente il proprio codice** ad ogni propagazione.

- **Meccanismo:** Utilizza un **motore metamorfico** (un vero e proprio mini-compilatore/assemblatore) integrato.
- **Tecniche di Mutazione:**
 - *Register Swapping:* Sostituisce i registri usati (es. `MOV EAX, 1` diventa `MOV EBX, 1` e usa `EBX` successivamente).
 - *Instruction Substitution:* Sostituisce istruzioni con equivalenti (es. `SUB EAX, EAX` per azzerare un registro, al posto di `XOR EAX, EAX` o `MOV EAX, 0`).
 - *Code Transposition:* Riordina blocchi di codice e funzioni, ricollegandoli con `JMP` e `CALL`.
 - *Garbage Code Insertion:* Inserisce codice "spazzatura" che non fa nulla (istruzioni NOP o calcoli inutili) per alterare la firma binaria.
- **Rilevamento:** Estremamente complesso. Poiché non c'è un payload statico da decifrare, l'emulazione è meno efficace. Richiede analisi semantica del codice o tecniche di N-gram analysis per identificare pattern statistici.

3.3 Difficoltà del Reverse Engineering del Malware

Il reverse engineering (analisi statica con disassemblatori come IDA Pro, Ghidra, e analisi dinamica con debugger come x64dbg, OllyDbg) è l'attività principale dell'analista di malware. I malware moderni sono progettati specificamente per contrastare questa attività.

3.3.1 Packing e Crittografia (Runtime Packers)

Molti malware sono "impacchettati" (packed). Il codice eseguibile originale è compresso e/o cifrato e inserito come risorsa o sezione dati all'interno di un *loader* (stub).

- **Analisi Statica:** Inutile. Il disassemblatore vede solo il codice del loader (es. UPX, Themida, VMProtect, o packer custom).
- **Analisi Dinamica:** L'analista deve eseguire il malware in un debugger, raggiungere l'Original Entry Point (OEP) – il punto in cui il codice è stato completamente decompresso in memoria – ed effettuare un *memory dump* per ottenere il payload originale su cui lavorare.

3.3.2 Tecniche Anti-Debugging

Il malware cerca attivamente di rilevare la presenza di un debugger.

- **API Checks:** Chiama funzioni come `IsDebuggerPresent()` o `CheckRemoteDebuggerPresent()`.
- **Timing Attacks:** Misura il tempo di esecuzione di alcune istruzioni (es. RDTSC). Se l'esecuzione è troppo lenta (a causa dei breakpoint e dello step-over del debugger), il malware termina.
- **Exception Handling:** Sfrutta il modo in cui i debugger gestiscono le eccezioni (es. SEH - Structured Exception Handling). Il malware può generare un'eccezione intenzionale e piazzare il proprio codice malevolo nell'handler, sapendo che un debugger intercetterà l'eccezione prima dell'handler.

3.3.3 Tecniche Anti-VM / Anti-Sandbox

Il malware cerca di capire se è in esecuzione in un ambiente virtualizzato (VMWare, VirtualBox) o in una sandbox (Cuckoo, ANY.run).

- **Artefatti Virtuali:** Cerca la presenza di file, driver o registry key specifiche (es. `VBoxGuestAdditions.sys`, `vmtoolsd.exe`).
- **MAC Address:** Controlla il MAC address della scheda di rete (le VM hanno OUI noti: 08:00:27 per VirtualBox, 00:05:69 per VMWare).
- **I/O Port Checks:** Utilizza istruzioni privilegiate o porte I/O usate dai driver di virtualizzazione per comunicare con l'hypervisor.
- **Sistemi "Troppo Puliti":** Controlla la cronologia del browser, i file recenti, o il numero di core della CPU. Una VM da analisi è spesso troppo "pulita" o ha poche risorse.

Se rileva un ambiente di analisi, il malware **termina l'esecuzione** o **esegue un payload benigno** (es. una "decoy") per ingannare l'analista.

4 Malware basati sul Payload

Questa categoria si concentra sullo *scopo* finale del malware.

4.1 Trojan (Cavallo di Troia)

Un **Trojan** è un malware che si maschera da software legittimo, utile o desiderabile (es. un gioco, un'utility, un aggiornamento software) per indurre l'utente a eseguirlo.

- **Funzionamento:** A differenza di virus e worm, **un Trojan non si replica da solo**. Dipende dall'ingegneria sociale.
- **Scopo:** È un **vettore di consegna** (*dropper* o *downloader*). Una volta eseguito, installa segretamente altri tipi di malware.

- **Tipi Comuni:**

- **RAT (Remote Access Trojan):** Fornisce all'attaccante un accesso completo e remoto al sistema (come un VNC o RDP nascosto).
- **Banking Trojan:** (Es. *Zeus*, *TrickBot*). Inietta codice nei browser web per rubare credenziali bancarie (spesso tramite *Man-in-the-Browser*).
- **Downloader:** Un piccolo Trojan il cui unico scopo è scaricare ed eseguire un payload più grande (es. ransomware) dal server C&C.

4.2 Backdoor

Una **Backdoor** (porta di servizio) è un meccanismo che bypassa i normali controlli di autenticazione per garantire all'attaccante un accesso persistente e nascosto a un sistema.

Può essere un programma installato appositamente (spesso da un Trojan o un Worm) o una debolezza intenzionale lasciata in un software legittimo da uno sviluppatore (per manutenzione o per scopi malevoli). Un RAT è una forma avanzata di backdoor.

4.3 Rootkit

Un **Rootkit** è un software (o un insieme di strumenti) progettato specificamente per **occultare la presenza** di un attacco (es. nascondere file, processi, connessioni di rete, e la presenza del rootkit stesso). Ottiene privilegi elevati (spesso a livello *root* o *kernel*).

- **User-Mode Rootkit:** Opera nello spazio utente. Utilizza tecniche come:
 - **API Hooking (IAT/EAT Hooking):** Modifica le tabelle di import/export (Import Address Table / Export Address Table) di librerie di sistema (es. `kernel32.dll`). Quando un programma legittimo (es. Task Manager) cerca di elencare i processi, la sua chiamata API viene intercettata e il rootkit filtra il proprio processo dall'elenco.
 - **DLL Injection:** Inietta la propria DLL in tutti i processi attivi per controllarli dall'interno.
- **Kernel-Mode Rootkit:** Opera a livello del kernel (Ring 0). È molto più potente e pericoloso.
 - **Driver Malevoli (LKM in Linux):** Si carica come un driver di dispositivo legittimo.
 - **DKOM (Direct Kernel Object Manipulation):** Modifica direttamente le strutture dati opache del kernel. Ad esempio, per nascondere un processo, rimuove il suo nodo (`EPROCESS`) dalla lista concatenata dei processi attivi mantenuta dal kernel.
 - **SSDT Hooking:** Modifica la *System Service Descriptor Table*, che mappa le chiamate API user-mode alle loro implementazioni kernel-mode.
- **Rilevamento:** I rootkit kernel-mode sono quasi impossibili da rilevare dall'interno del sistema operativo infetto (poiché controllano le API usate per rilevarli). Richiedono scanner offline (eseguiti da un CD/USB di avvio) o analisi della memoria (memory forensics).

4.4 Ransomware

Un **Ransomware** è un malware che monetizza l'attacco tramite estorsione.

- **Locker Ransomware (obsoleto):** Blocca l'accesso al sistema (es. mostra una schermata a pieno schermo) ma non cifra i dati. Facilmente aggirabile.

- **Crypto-Ransomware (moderno):** Cifra i file dell'utente (documenti, foto, database) utilizzando tecniche crittografiche robuste (es. una combinazione di AES-256 e RSA-2048).

Processo di Infezione (Crypto-Ransomware):

1. **Consegna:** Tramite phishing, exploit kit, o worm.
2. **Generazione Chiavi:** Il malware contatta il server C&C per ottenere una chiave pubblica (RSA) o genera una coppia di chiavi sul posto (cifrando la privata con una chiave pubblica master).
3. **Cifratura:** Per velocità, il malware genera una chiave simmetrica (es. AES-256) per ogni file. Cifra il file con AES e poi cifra la chiave AES con la chiave pubblica RSA dell'attaccante. Il risultato (chiave AES cifrata) viene salvato nel file o in un file di metadati.
4. **Distruzione Backup:** Cerca ed elimina le copie shadow (`vssadmin delete shadows /all /quiet`) e altri backup locali.
5. **Riscatto:** Rilascia una nota (*ransom note*) che spiega che i file sono cifrati e che l'unico modo per recuperarli è pagare un riscatto (di solito in Bitcoin) per ottenere la chiave privata RSA (o un decryptor).

4.4.1 Caso di Studio: WannaCry

WannaCry (2017) è stato un attacco ibrido che combinava un **Ransomware** con un **Worm**.

- **Vettore di Propagazione (Worm):** Utilizzava l'exploit **EternalBlue** (rubato all'NSA e rilasciato dal gruppo Shadow Brokers). Questo exploit colpiva una vulnerabilità critica (MS17-010) nel protocollo **SMBv1** di Windows.
- **Meccanismo:** Una volta infettato un sistema, WannaCry cifrava i file (payload Ransomware) e contemporaneamente scansionava la rete (sia LAN che Internet) alla ricerca di altre macchine Windows vulnerabili sulla porta 445 (SMB) per diffondersi (payload Worm).
- **Impatto:** La sua natura di worm gli ha permesso di diffondersi a livello globale in poche ore, colpendo centinaia di migliaia di computer, inclusi sistemi critici come ospedali (NHS nel Regno Unito), trasporti e aziende.
- **Kill Switch:** L'attacco è stato rallentato dalla scoperta di un "kill switch" (un dominio hardcoded che il malware cercava di contattare; se la connessione riusciva, il malware terminava l'esecuzione).

4.5 Spyware e Keylogger

Lo **Spyware** è un software progettato per raccogliere informazioni sull'utente, sulle sue abitudini o sulle sue credenziali senza consenso.

- **Keylogger:** Un tipo specifico di spyware che registra ogni tasto premuto dall'utente. È il metodo più diretto per rubare password, conversazioni e dati di carte di credito. Può essere implementato via software (hooking delle API `GetAsyncKeyState`) o hardware (un dispositivo fisico tra tastiera e PC).
- **InfoStealer:** Sottrae informazioni salvate (es. password e cookie dai browser, file di configurazione di client FTP o wallet di criptovalute).

4.6 Logic Bomb (Bomba Logica)

Una **Logic Bomb** è un frammento di codice malevolo inserito intenzionalmente in un software, che rimane dormiente finché non si verifica una specifica **condizione di attivazione** (trigger).

- **Trigger:** Può essere una data o un'ora specifica (es. il venerdì 13), il verificarsi o il non verificarsi di un evento (es. l'esecuzione di un programma), o la presenza/assenza di un file (es. un file che controlla se un dipendente è ancora presente nei sistemi aziendali).
- **Contesto:** Sono quasi sempre opera di **insider** (es. un amministratore di sistema scontento che prepara una bomba per cancellare i server se il suo account viene disattivato).
- **Differenza dal Payload:** A differenza del payload di un virus (come CIH, che si attiva in una data specifica), la logic bomb è tipicamente un attacco mirato e non progettato per la replicazione.

5 Malware Moderni: Fileless

Una tendenza critica nell'evoluzione del malware è l'approccio **Fileless** (o "Living-off-the-Land" - LotL). Questi attacchi cercano di non scrivere alcun file eseguibile malevolo sul disco, eludendo così la scansione degli antivirus tradizionali (che si basano sull'analisi dei file).

- **Meccanismo:** Il malware opera interamente **in-memory**, cioè non scarica mai il virus ma lo carica direttamente in **RAM** in modo da non essere detectabile.
- **Vettori:** Sfrutta strumenti legittimi e già presenti nel sistema operativo per entrare nel sistema:
 - **PowerShell:** Estremamente potente. Può scaricare script dalla rete ed eseguirli direttamente in memoria (`IEX (New-Object Net.WebClient).DownloadString('...')`).
 - **WMI (Windows Management Instrumentation):** Può essere usato per l'esecuzione di comandi e la persistenza (WMI event subscriptions).
 - **Macro (VBA):** Macro malevole in documenti Office che scaricano ed eseguono script in memoria.
 - **Registro di Sistema:** Usato per salvare il payload, che viene poi letto ed eseguito da un altro script.
- **Rilevamento:** Molto difficile. Richiede un monitoraggio comportamentale avanzato (EDR - Endpoint Detection and Response), analisi della memoria, e un logging granulare degli script eseguiti (es. PowerShell Script Block Logging).